

Process & Decision Documentation

Side Quests and A4 (Individual Work)

Focus on:

- Changing the example code to reflect an interactive art gallery game.
- Adding background images
- Adding a smiley face and frowny face to the win / lose screens along with points every time there is a win (someone purchase's the artist's painting)

GenAI Documentation

Date Used: February 1, 2026

Tool Disclosure: M365 Copilot

Purpose of Use: To help code my interactive story game ideas and debugging.

Summary of Interaction: I began with telling Copilot my ideas for this side quest and asking if it could assist me with ways on how I could code it. I also used the GenAI as a tool to help me debug issues I was having with my code like asking it why my background image wasn't showing when I ran the code or asking it questions about the code it suggested me to use.

Human Decision Point(s): After the smiley face was drawn, the text on the win screen was drawn on top of it and the text was no longer readable. After analyzing the code myself, I noticed that it was adding a stroke to the text because of the code above it. To fix this I added a `noStroke()` code after the text's fill and moved the text above, so it doesn't overlap the smiley face. Similarly, I applied this type of human decision to the GenAI output throughout the coding process like moving the text location and the rectangle text background I added on the main game screen, so the text doesn't overlap the image I put in the background and it was still readable.

Integrity & Verification Note: I checked GenAI output for accuracy by opening the live server to see my sketch and if my codes aligned with my interactive story vision. I made

sure each time I saved a new line of code that the game was still interactive and running correctly.

Scope of GenAI Use: GenAI did not contribute to the idea of my changes. I thought about the story I wanted my game to tell and the changes I wanted to add. The idea of having an interactive art gallery where users press the button and have an equal opportunity to get coins if a visitor randomly decides to purchase their art piece. Along the UI decision of adding a photo as the background.

Limitations or Misfires: The GenAI couldn't account for the background image I used because I didn't give them the file, so by default the text was not aligned properly and the text over the background made it hard to read.

Summary of Process (Human + Tool)

At first, I wasn't sure how to proceed with this side quest so I asked the GenAI for ideas to see the options they would have and came up with my own idea of an interactive art gallery. I went back and forth with the GenAI asking it how to complete different parts of the code to make my interactive story game come to life. I would ask it questions like how to add a photo as my background, why my code wasn't working, or any other questions I had like where I should insert specific codes. I would try the code, run it on the open live server and make my own iterations. Sometimes text would be overlapping, text wasn't readable, or the UI was just not aesthetically pleasing, so I chose to make changes to the design or my ideas.

Appendix

Please include a full transcript of your conversation with the GenAI. GenAI transcripts are used for transparency and verification only and are not assessed for writing quality or completeness.

Note: If GenAI is used in another language, you must include both the original language and a translated transcript in the appendix

Can you give me some ideas for a game that has a start screen with 2 buttons that gives the users 2 different options?

Absolutely — a two-button start screen is a great design constraint because it forces clarity and creativity. You can build a surprising number of game concepts around just two choices. Here are several directions you could take, each with a distinct vibe and gameplay loop.

Concept 1: “Light or Shadow” — A Dual-Path Adventure

Buttons: *Light / Shadow* Each button launches a completely different storyline, environment, and ability set.

- *Light path:* healing, protection, puzzle-solving
- *Shadow path:* stealth, sabotage, risk-reward mechanics Players can replay the game to see how the two paths intersect or diverge.

Concept 2: “Think or Act” — Strategy vs. Reflexes

Buttons: *Strategy Mode / Arcade Mode*

- Strategy mode: turn-based challenges, planning, resource management
- Arcade mode: fast-paced action, time-based obstacles Same world, two ways to experience it.

Concept 3: “Predator or Prey” — Survival Game

Buttons: *Predator / Prey*

- Predator: hunt, track, and outsmart faster creatures
- Prey: hide, escape, and use the environment to survive Two perspectives on the same ecosystem.

Concept 4: “Create or Destroy” — Physics Sandbox

Buttons: *Build / Break*

- Build mode: construct structures, machines, or contraptions
- Break mode: destroy them with tools, physics, or natural disasters
- Players can switch between modes after unlocking them.

Concept 5: “Truth or Lie” — Detective Narrative

Buttons: *Tell the Truth / Tell a Lie* Each choice affects the story, relationships, and outcomes. The game becomes a branching narrative where honesty and deception shape the world.



Concept 6: “Escape or Explore” — Open-World Mini Game

Buttons: *Escape Mode / Explore Mode*

- Escape: timed challenges, chase sequences
- Explore: relaxed wandering, collectibles, lore discovery Two moods in one game.



Concept 7: “Magic or Might” — RPG Starter Choice

Buttons: *Magic / Might* This determines the player’s class, abilities, and early quests. A classic but effective way to set tone and replay value.



Concept 8: “Experiment or Observe” — Science Puzzle Game

Buttons: *Experiment / Observe*

- Experiment: hands-on puzzles where players mix, test, and manipulate elements
- Observe: deduction-based puzzles where players analyze patterns and clues Two complementary ways to solve the same mystery.



Concept 9: “Random” vs. “Skill” — Challenge Game

Buttons: *Random Mode / Skill Mode*

- Random: unpredictable modifiers, chaotic gameplay
- Skill: precise, consistent challenges Players choose chaos or mastery.



Concept 10: “Hero” or “Villain” — Morality-Driven RPG

Buttons: *Be the Hero / Be the Villain*

Two campaigns with mirrored goals, characters, and moral choices.

If you want, I can help you:

- pick one and expand it into a full game design
- sketch out the UI for the start screen
- write the code for the menu in Unity, Godot, or another engine
- brainstorm more concepts in a specific genre

Which direction feels closest to what you’re imagining?

I was thinking of an art game that incorporates an art gallery. But i'm not sure how to build an interactive story around it where the user starts with the option to pick between 2 buttons on the home screen

An art-gallery game is a beautiful foundation — and the two-button start screen can become the *core narrative hook* rather than just a menu. The trick is to make those two buttons represent two **perspectives**, **timelines**, or **roles** that shape how the player experiences the gallery.

Below are several story structures that fit an art-game vibe and turn the two-button choice into something meaningful and replayable.

1. “Artist or Visitor” — Two Sides of the Same Gallery

Buttons: *Enter as the Artist / Enter as the Visitor*

As the Artist

- You walk through the gallery seeing your own works.
- Each piece triggers memories, sketches, voice notes, or unfinished ideas.
- The story becomes a reflection on creation, doubt, and growth.

As the Visitor

- You interpret the same artworks without context.
- You form your own theories, misunderstandings, or emotional reactions.
- The story becomes about perception and meaning.

The twist: The two paths slowly reveal how different people experience the same art — and the truth lies somewhere between them.

2. “Past or Present” — A Gallery That Changes Over Time

Buttons: *Past Exhibition / Present Exhibition*

Past Exhibition

- The gallery is full of early works, sketches, and abandoned concepts.
- The player sees the artist’s struggles, experiments, and failures.

Present Exhibition

- The gallery shows polished, curated pieces.
- The player sees what the world *thinks* the artist is.

The story emerges from comparing what was once hidden with what is now displayed.

3. “Reality or Imagination” — A Gallery That Comes Alive

Buttons: *Reality / Imagination*

Reality

- A quiet, minimalist gallery.
- You read plaques, listen to audio guides, and learn about the artist.

Imagination

- The artworks animate, speak, or transform.
- You enter surreal mini-worlds inside each painting.

The narrative tension: Which version reflects the “true” meaning of the art?

4. “Truth or Lie” — A Mystery Hidden in the Gallery

Buttons: *Believe the Curator / Question the Curator*

Believe the Curator

- You follow the official story behind each artwork.
- Everything seems orderly and respectable.

Question the Curator

- You uncover contradictions, hidden messages, or altered pieces.
- The gallery becomes a puzzle box of secrets.

Replay value: Players compare the two narratives to uncover what really happened.

5. “Create or Observe” — A Game About Artistic Process

Buttons: *Create / Observe*

Create

- You start in a studio connected to the gallery.
- Each artwork is a puzzle or mini-game representing a creative challenge.

- Completing them “adds” the piece to the gallery.

Observe

- You walk through the gallery seeing the finished works.
- You learn how each piece was made — or misinterpreted.

The emotional arc: Art is both the messy process and the polished result.

🌙 6. “Day or Night” — Two Moods, Two Stories

Buttons: *Daytime Gallery / Nighttime Gallery*

Daytime

- Visitors, noise, commentary, social pressure.
- You overhear conversations and see how people judge the art.

Nighttime

- The gallery is empty.
- The art whispers, glows, or reveals hidden layers.

The theme: Art changes depending on when and how you look at it.

If you want, I can help you:

- pick one of these and expand it into a full narrative
- design the start screen visually
- outline the player’s journey room by room
- write the story beats for each button
- brainstorm the artworks themselves

Which direction feels closest to the game you want to make?

how do I add an image in the background of the start screen?

You can absolutely add a background image to your start screen in p5.js — and it’s pretty simple once you know the pattern. The key steps are:

1. **Load the image in** `preload()`
2. **Draw it in the background during the start screen state**

Here's how to integrate it cleanly into the sketch you already have.

Add a Background Image to the Start Screen

1. Put your image file in the same folder as your sketch

For example:

Code

`background.jpg`

2. Add a `preload()` function to load the image

javascript

`let startBg;`

```
function preload() {  
  startBg = loadImage("background.jpg");  
}
```

3. Draw the image in your start screen

Replace your `drawStartScreen()` with this:

javascript

```
function drawStartScreen() {  
  // Draw the background image  
  image(startBg, 0, 0, width, height);  
  
  // Title text  
  textAlign(CENTER);  
  textSize(32);  
  fill(255); // white text so it shows on dark backgrounds
```



```
text("Interactive Art Gallery", width / 2, height / 2 - 80);

textSize(16);

text("Choose your role to begin", width / 2, height / 2 - 40);
}
```

✖ Full Updated Structure (only the new parts)

javascript

```
let startBg;
```

```
function preload() {
  startBg = loadImage("background.jpg");
}
```

```
function drawStartScreen() {
  image(startBg, 0, 0, width, height);
```

```
  textAlign(CENTER);
  textSize(32);
  fill(255);
  text("Interactive Art Gallery", width / 2, height / 2 - 80);

  textSize(16);
  text("Choose your role to begin", width / 2, height / 2 - 40);
}
```

★ Tips to Make It Look Good

- If your image is too bright, you can overlay a semi-transparent tint:

javascript

```
tint(255, 180); // slightly fade the image
```

```
image(startBg, 0, 0, width, height);
```

```
noTint();
```

- If your text blends into the background, add a shadow:

javascript

```
fill(0, 150);
```

```
text("Interactive Art Gallery", width / 2 + 2, height / 2 - 78);
```

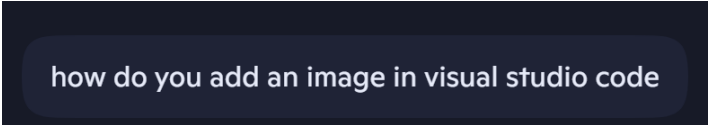
```
fill(255);
```

```
text("Interactive Art Gallery", width / 2, height / 2 - 80);
```

If you want, I can help you add:

- a fade-in animation for the start screen
- animated buttons
- a gallery-themed background
- different backgrounds for Artist vs Visitor paths

Just tell me what direction you want to take next!



how do you add an image in visual studio code

Adding an image in **Visual Studio Code** isn't about the editor itself — it's about putting the image file in the right place in your project so p5.js can load it.

Here's the simplest way to do it.



How to Add an Image to Your p5.js Project in VS Code

1. Put your image file in your project folder

Inside your project, you should have something like:

Code

project-folder/

```
|
|
|— sketch.js
|— index.html
|— background.jpg ← your image goes here
└— other files...
```

You can add the image by:

- dragging it into the folder in VS Code's sidebar
- or copying it into the folder using your file explorer

2. Load the image in your p5.js sketch

Use `preload()` so p5 loads the image before the sketch starts.

javascript

```
let startBg;
```

```
function preload() {
  startBg = loadImage("background.jpg");
}
```

If your image is inside a subfolder (like `assets/`), include the folder name:

javascript

```
startBg = loadImage("assets/background.jpg");
```

3. Draw the image in your start screen

Use `image()` inside your drawing function:

javascript

```
function drawStartScreen() {
  image(startBg, 0, 0, width, height);
}
```

★ Common Mistakes to Avoid

✖ Wrong file path

If your image is in a folder, you must include the folder name.

✖ Wrong file name

File names are case-sensitive:

- Background.jpg ≠ background.jpg

✖ Forgetting to use `preload()`

Images won't load correctly if you try to load them in `setup()`.

If you want, I can help you:

- organize your project folder
- add multiple images
- add clickable artwork images in the gallery
- animate or fade in your background image

Just tell me what you want to build next!

```

let backgroundimg;

function preload() {
  backgroundimg = loadImage("backgroundimg.jpg");
}

// drawStart() is called from main.js only when:
// currentScreen === "start"
function drawStart() {
  // Background colour for the start screen
  background(180, 225, 220); // soft teal background
  Image(backgroundimg, 0, 0, width, height);

  // ---- Title text ----
  fill(30, 50, 60);
  textSize(46);
  textAlign(CENTER, CENTER);
  text("Interactive Art Gallery", width / 2, 180);

  // ---- Buttons (data only) ----
  // These objects store the position/size/label for each button.
  // Using objects makes it easy to pass them into drawButton()
  // and also reuse the same information for hover checks.
  const startBtn = {
    x: width / 2,
    y: 320,
    w: 240,
    h: 80,
    label: "START",
  }; why isn't the background image showing in the background?

```

You're very close — the issue comes down to one small but important detail:

✅ **You wrote `Image()` instead of `image()`**

JavaScript is case-sensitive, and in p5.js the function must be **lowercase**:

js

`image(backgroundimg, 0, 0, width, height);`

Using `Image()` (capital I) calls **nothing**, so the image never draws.

✅ **Corrected version of your code**

javascript

let backgroundimg;

```
function preload() {  
  backgroundimg = loadImage("backgroundimg.jpg");  
}  
  
function drawStart() {  
  // Draw background image FIRST  
  image(backgroundimg, 0, 0, width, height);  
  
  // Optional: overlay a tint or background color if you want  
  // background(180, 225, 220, 100);  
  
  // ---- Title text ----  
  fill(30, 50, 60);  
  textSize(46);  
  textAlign(CENTER, CENTER);  
  text("Interactive Art Gallery", width / 2, 180);  
  
  // ---- Buttons ----  
  const startBtn = {  
    x: width / 2,  
    y: 320,  
    w: 240,  
    h: 80,  
    label: "START",  
  };  
}
```

Other things to double-check if it still doesn't show

1. Is the image file in the correct folder?

It must be in the same folder as your sketch unless you specify a path:

Code

```
project/  
| sketch.js  
| main.js  
| backgroundimg.jpg ← must be here
```

If it's in a folder:

```
js  
  
backgroundimg = loadImage("assets/backgroundimg.jpg");
```

2. Is the filename EXACT?

backgroundimg.jpg ≠ BackgroundImg.JPG Case and extension must match.

3. Are you drawing the image *after* calling background()?

If you do this:

```
js  
  
background();  
  
image();
```

The image appears.

If you do this:

```
js  
  
image();  
  
background();
```

The background covers the image.

Your code is correct in this regard — just keep the image first if you want it visible.

If you want, I can help you add:

- a fade-in effect for the background
- a semi-transparent overlay so text is easier to read
- animated buttons
- different backgrounds for Artist vs Visitor

Just tell me what direction you want to take next!

```
const gameBtn = {
  x: 400, // x position (centre of the button)
  y: 550, // y position (centre of the button)
  w: 260, // width
  h: 90, // height
  label: "PRESS HERE", // text shown on the button
};

let artimg;

function preload() {
  artimg = loadImage("art piece.jpg"); // Photo by Simona Sergi on
  Unsplash https://unsplash.com/photos/man-in-black-jacket-standing-in-
  front-of-white-wall-with-orange-and-yellow-heart-wall-OfjOuw_prPg
}

// -----
// Main draw function for this screen
// -----
// drawGame() is called from main.js "only"
// when currentScreen === "game"
function drawGame() {
  // Set background colour for the game screen
  image(artimg, 0, 0, width, height);

  // ---- Title and instructions text ----
  fill(0); // black text
  textSize(32);
  textAlign(CENTER, CENTER);
  text("Game Screen", width / 2, 160);

  textSize(18);
  text(
    "Click the button (or press ENTER) for a random result.",
    width / 2,
    210,
  );
  // why is the code not showing? I also have a preload() function in the
  // other screen. Is that why?
```

Your code is valid JavaScript, but the **reason it's not showing on your webpage** is almost certainly **not** because of the snippet itself — it's because of **how p5.js handles preload()** and **how your project is structured across multiple screens/files**.

Let's break down the likely causes so you can pinpoint the issue.

✅ **Most likely cause: You have *two* preload() functions**

In p5.js, **only one preload() function is allowed**.

If you define preload() in multiple files, the **last one loaded overwrites the earlier one**, meaning:

- The image in your game screen **never loads**
- artimg stays **undefined**
- image(artimg, 0, 0, width, height) silently fails
- Your text still draws, but the background image does not

This is the #1 cause when people use multi-screen setups.

✅ **How to fix it**

You have two options:

Option 1 — Use ONE global preload()

Put *all* your asset loading in a single preload() function:

js

let artimg;

let otherImg;

```
function preload() {  
  artimg = loadImage("art piece.jpg");  
  otherImg = loadImage("otherScreenImage.png");  
}
```

Then each screen just uses the images — no more preload() inside screen files.

Option 2 — Use p5's loadImage() inside setup() or draw()

If you want each screen to load its own assets, don't use preload() at all:

js

```
function drawGame() {  
  if (!artimg) {  
    artimg = loadImage("art piece.jpg");  
  }  
  image(artimg, 0, 0, width, height);  
}
```

This avoids preload() conflicts.

⚠ Other possible issues (less likely but worth checking)

1. Wrong file path

If "art piece.jpg" is not in the same folder as your sketch, it won't load.

2. The image is too large

Huge images sometimes fail silently depending on browser memory.

3. drawGame() is not being called

Make sure your main.js actually switches to "game".

🎯 Quick test

Add this inside drawGame():

```
js
```

```
console.log(artimg);
```

If it prints **undefined**, your preload() is being overwritten.

If you want, paste your file structure or the other preload() function and I can tell you exactly where the conflict is.

does preload go before setup?

In p5.js, **yes — preload() always runs *before* setup()**, no matter where the functions appear in your code file.

But here's the important part: **The order they *run* is fixed, but the order you *write* them in the file does not matter.**

p5.js calls them in this sequence:

1. **preload()**
2. **setup()**
3. **draw()** (loops)

So even if your file looks like this:

```
js
```

```
function setup() { ... }
```

```
function draw() { ... }
```

```
function preload() { ... }
```

p5.js will still run `preload()` first.

Why this matters for your project

Because `preload()` runs first, p5.js expects **only one `preload()`** in your entire sketch. If you define a second `preload()` in another file, the last one loaded overwrites the first.

That's why your image wasn't showing earlier — your `preload()` in the other screen replaced the one that loads `artimg`.

If you want, I can help you restructure your multi-screen setup so all your assets load correctly without conflicts.

```

function drawGame() {
  // Set background colour for the game screen
  image(artimg, 0, 0, width, height);

  fill(0);
  rect(2000, 300, 80);
  rectMode(CENTER);

  // ---- Title and instructions text ----
  fill(255); // white text
  textSize(32);
  textAlign(CENTER, CENTER);
  text("Welcome to the Interactive Art Gallery", width / 2, 160);

  textSize(18);
  text(
    "Click the button (or press ENTER) to see if the visitor chooses to",
    "buy your art piece.",
    width / 2,
    210,
  ); how do I add the rectangle to be big enough and at the right
  location for it to be behind the title and instructions text?
}

```

You're very close — the only issue is **your rectangle is drawn in the wrong place, with the wrong size, and before you set** `rectMode(CENTER)`.

Here's the clean way to place a background rectangle *behind* your title + instructions.

✅ Step 1 — Move `rectMode(CENTER)` before drawing the rectangle

Right now you call:

```

js
fill(0);

rect(2000, 300, 80);

rectMode(CENTER);

```

That means the rectangle is drawn using the **default mode (CORNER)**, not `CENTER`.

✅ Step 2 — Choose a rectangle size that covers both text lines

Your text is around:

- Title at **y = 160**
- Instructions at **y = 210**

So a rectangle centered around **y ≈ 185** with height ~120 works well.

✅ Step 3 — Draw the rectangle *before* the text

That way the text appears on top.

Updated version that works

js

```
function drawGame() {  
  image(artimg, 0, 0, width, height);  
  
  // Draw background rectangle behind text  
  rectMode(CENTER);  
  fill(0, 150); // semi-transparent black looks nice  
  rect(width / 2, 185, 700, 120);  
  
  // x = center, y = between the two text lines  
  // width = 700px, height = 120px (adjust as needed)  
  
  // ---- Title and instructions text ----  
  fill(255);  
  textSize(32);  
  textAlign(CENTER, CENTER);  
  text("Welcome to the Interactive Art Gallery", width / 2, 160);  
  
  textSize(18);  
  text(  
    "Click the button (or press ENTER) to see if the visitor chooses to buy your art piece.",  
    width / 2,  
    210  
  );  
}
```

Why this works

- The rectangle is centered behind both text lines.
- It's drawn *before* the text, so the text appears on top.
- `rectMode(CENTER)` ensures the rectangle expands evenly around the center point.
- A semi-transparent fill makes the text readable without blocking the background art.

If you want, I can help you fine-tune the size, add rounded corners, or animate the panel.

can I move the text and rectangle a bit higher?

Absolutely — you can shift both the rectangle **and** the text upward just by adjusting their **y-positions**. In p5.js, smaller y-values move things higher on the canvas.

Here's a clean version with everything nudged upward:

js

```
function drawGame() {  
  image(artimg, 0, 0, width, height);  
  
  // Background rectangle behind text  
  rectMode(CENTER);  
  fill(0, 150);  
  rect(width / 2, 150, 700, 120);  
  // moved from y=185 → y=150  
  
  // Title text  
  fill(255);  
  textSize(32);  
  textAlign(CENTER, CENTER);  
  text("Welcome to the Interactive Art Gallery", width / 2, 120);  
  // moved from y=160 → y=120
```

```
// Instructions text
textSize(18);
text(
  "Click the button (or press ENTER) to see if the visitor chooses to buy your art piece.",
  width / 2,
  170
);
// moved from y=210 → y=170
}
```

How to adjust it yourself

- Move everything **up** → decrease the y-values
- Move everything **down** → increase the y-values
- Keep the rectangle centered between the two text lines

If you want, I can help you align them perfectly or add padding so the rectangle always fits the text automatically.

P5.js code of a smiley face in the center of the page that I could use on the win screen

Here's a clean, self-contained p5.js snippet that draws a smiley face perfectly centered on the canvas — ideal for your win screen. You can drop this directly into your drawWin() function or wherever you handle that screen.

Centered Smiley Face (p5.js)

```
js
function drawWin() {
  background(255); // white background
```

```
// Center coordinates

let cx = width / 2;

let cy = height / 2;


// Face

fill(255, 220, 0); // yellow

noStroke();

ellipse(cx, cy, 250, 250);


// Eyes

fill(0);

ellipse(cx - 50, cy - 40, 30, 40); // left eye
ellipse(cx + 50, cy - 40, 30, 40); // right eye


// Smile

noFill();

stroke(0);

strokeWeight(8);

arc(cx, cy + 20, 150, 120, 0, PI); // smiling mouth
}
```



How it works

- cx and cy keep everything centered no matter the canvas size.
- The face is a 250px yellow circle.
- Eyes are black ellipses positioned relative to the center.
- The smile is a simple arc drawn below the eyes.

If you want, I can help you:

- animate the smiley
- add sparkles or confetti
- make the smiley bounce or spin
- match the colors to your game's theme

Just tell me the vibe you want for your win screen!

how would I write the arc to turn the smile into a frown

To turn your smile into a frown, you only need to **flip the arc upside-down** by changing the start and end angles.

In p5.js:

- A **smile** uses: `arc(cx, cy + 20, 150, 120, 0, PI);`
- A **frown** uses the opposite angles: `arc(cx, cy + 60, 150, 120, PI, TWO_PI);`

Here's the frown version:

js

// Frown

`noFill();`

`stroke(0);`

`strokeWeight(8);`

`arc(cx, cy + 60, 150, 120, PI, TWO_PI);`

What changed

- **Start angle:** from 0 → PI
- **End angle:** from PI → TWO_PI
- **Y-position:** moved slightly *down* (cy + 60) so the frown sits below the eyes

If you want the frown to be more dramatic, you can increase the height or shift it further down.

what is PI

In p5.js (and in most programming languages), **PI** is a built-in constant that represents the mathematical value of π — about **3.14159265....**

You use it whenever you're working with angles, circles, arcs, or rotations.

A quick way to think about it

- A full circle = **TWO_PI** (≈ 6.28318)
- Half a circle = **PI** (≈ 3.14159)
- Quarter circle = **HALF_PI** (≈ 1.5708)

Why your smile/frown uses PI

p5.js draws arcs using radians, not degrees.

So:

- **Smile:** `arc(cx, cy, w, h, 0, PI)` → draws the *bottom* half of a circle
- **Frown:** `arc(cx, cy, w, h, PI, TWO_PI)` → draws the *top* half of a circle

If you ever want to switch to degrees, you can call:

js

`angleMode(DEGREES);`

But for most p5.js sketches, sticking with radians (PI, HALF_PI, etc.) keeps things simple.

If you want, I can show you how to animate the smile turning into a frown

How would I code a screen that shows the players points they've earned after each win

You can absolutely build a clean “points earned” screen in p5.js — and it's easier than you might think. The key is to keep track of the player's score in a **global variable**, update it whenever they win, and then draw a dedicated screen that displays it.

Here's a simple, flexible pattern you can drop into your existing multi-screen setup.

🌟 1. Create a global score variable

Put this at the top of your sketch (outside any function):

```
js
```

```
let playerPoints = 0;
```

🌟 2. Add points when the player wins

Wherever your game logic detects a win, update the score:

```
js
```

```
playerPoints += 10; // or whatever amount you want
```

```
currentScreen = "win"; // switch to the win screen
```

🌟 3. Create the win screen function

This draws a clean, centered “points earned” screen:

```
js
```

```
function drawWin() {
```

```
    background(240);
```

```
    textAlign(CENTER, CENTER);
```

```
    // Title
```

```
    fill(0);
```

```
    textSize(40);
```

```
    text("You Won!", width / 2, 150);
```

```
    // Points display
```

```
    textSize(28);
```

```
    text("Points Earned:", width / 2, 250);
```

```
    fill(50, 150, 255);
```

```
    textSize(60);
```

```
text(playerPoints, width / 2, 330);

// Instructions
fill(0);
textSize(20);
text("Press ENTER to play again", width / 2, 420);
}
```

🌟 4. Make sure your main draw() calls the right screen

In your main file:

```
js
function draw() {
  if (currentScreen === "game") {
    drawGame();
  } else if (currentScreen === "win") {
    drawWin();
  }
}
```

🌟 5. Restart the game when ENTER is pressed

```
js
function keyPressed() {
  if (keyCode === ENTER && currentScreen === "win") {
    currentScreen = "game";
  }
}
```

🎨 Result

Every time the player wins:

- Their score increases
- The win screen appears
- The total points are displayed
- They can press ENTER to play again

It's clean, readable, and fits perfectly with your multi-screen structure.

If you want, I can help you add animations, a smiley face, sound effects, or even a leaderboard.

How would I code the players points they've earned after each win in the win screen with the smiley face?

You can slot the player's points into your **existing win screen** with the smiley face very cleanly. You already have the global score variable and the win screen structure, so it's just a matter of drawing the text in the right place.

Here's a polished version that blends nicely with your smiley-face layout.

★ Add this inside your drawWin() function

js

```
function drawWin() {  
  background(255);  
  
  // Center of canvas  
  let cx = width / 2;  
  let cy = height / 2;  
  
  // Smiley face  
  fill(255, 220, 0);  
  noStroke();  
  ellipse(cx, cy, 250, 250);
```

```
fill(0);

ellipse(cx - 50, cy - 40, 30, 40);
ellipse(cx + 50, cy - 40, 30, 40);


noFill();

stroke(0);

strokeWeight(8);
arc(cx, cy + 20, 150, 120, 0, PI);


// --- POINTS TEXT ---

noStroke();

fill(0);

textAlign(CENTER, CENTER);


textSize(40);

text("You Won!", cx, cy - 180);


textSize(28);

text("Points Earned:", cx, cy + 160);


fill(50, 150, 255);

textSize(60);

text(playerPoints, cx, cy + 230);


// Instructions
```

```
fill(0);  
textSize(20);  
text("Press ENTER to continue", cx, cy + 300);  
}
```

★ What this gives you

- A smiley face centered on the screen
- A clear “You Won!” title above it
- A **Points Earned** section below the smiley
- The player’s actual score displayed in a large, readable font
- A prompt to continue

It keeps the win screen cheerful and readable without cluttering the smiley.

★ Make sure your score updates when the player wins

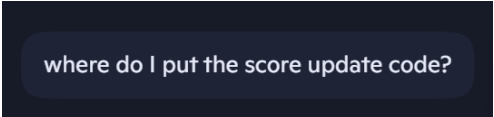
Wherever your win logic happens:

js

```
playerPoints += 10; // or whatever amount you want
```

```
currentScreen = "win";
```

If you want, I can help you animate the points (like a little pop-in effect) or add a sound cue when the player wins.



where do I put the score update code?

Great question — and this is one of those things that becomes super easy once you know *where* your game decides a win has happened.

The score update **must happen at the exact moment the player wins**, *before* you switch to the win screen.

Think of it like this:

1. Your game logic checks if the player won
2. If yes → **add points**

3. Then → switch to "win" screen

4. The win screen simply *displays* the updated score

Let me show you exactly where this goes.

★ Where to put the score update

Wherever your game currently says something like:

```
js
```

```
currentScreen = "win";
```

you want to update the score **right before** that line.

For example:

```
js
```

```
if (playerChoseCorrectly) {
```

```
    playerPoints += 10; // <-- UPDATE SCORE HERE
```

```
    currentScreen = "win";
```

```
}
```

Or if your win condition is random:

```
js
```

```
if (result === "win") {
```

```
    playerPoints += 10; // <-- ADD POINTS HERE
```

```
    currentScreen = "win";
```

```
}
```

Or if you use a button:

```
js
```

```
function mousePressed() {
```

```
    if (currentScreen === "game" && isHover(gameBtn)) {
```

```
        let result = random(["win", "lose"]);
```



```
if (result === "win") {  
    playerPoints += 10; // <-- SCORE UPDATE  
    currentScreen = "win";  
} else {  
    currentScreen = "lose";  
}  
}  
}
```

★ Why it goes here

- The **win screen should never modify the score**
- The win screen should only **display** the score
- The score should update **once per win**, not every frame

This keeps your logic clean and prevents accidental double-counting.

★ Your win screen then just reads the score

Inside drawWin() you simply show:

js

```
text(playerPoints, cx, cy + 230);
```

No updating here — just displaying.

If you want, you can show me your win-checking code and I'll point to the exact line where the score update belongs